Criar um bot Discord com JavaScript

Guia técnico do Último Round

Preparado para Samantha em 19 de outubro de 2026. Este guia acompanha o código do Último Round: um bot para simular clutches de VALORANT e consultar jogos e resultados competitivos. Explica a aplicação Discord, as credenciais, o fluxo das interações e os pontos do código que podes alterar. O servidor de testes escolhido é Bots Playground.

Esta primeira versão usa JavaScript ESM, Node.js 22.13 ou superior, discord.js 14.27.0 e Cheerio 1.2.0. Os comandos são /clutch, /jogos, /resultado e /ajuda. Não inclui IA, leitura de conversas, notícias automáticas ou pesquisa livre sobre campeonatos. O teste numa conta real exige configuração local; os testes de código não precisam de token.

1 Como o bot comunica com o Discord

O processo Node abre uma ligação WebSocket ao Gateway do Discord. Quando alguém executa um slash command, o Discord entrega um evento InteractionCreate. O handler identifica o comando, lê as opções e envia a resposta. O discord.js trata da ligação, heartbeats, reconexão e chamadas REST habituais.

O registo de comandos é uma operação separada: envia ao Discord o nome, a descrição e as opções de cada comando. Alterar uma descrição em commands.js não atualiza o Discord até executares novamente o script de registo. Alterar a lógica de um comando exige reiniciar o processo do bot, mas não exige novo registo se o contrato não mudou.

O Discord também suporta interações por HTTP, mas este projeto escolhe Gateway. Por isso não precisa de servidor HTTP público, túnel, porta aberta nem Interactions Endpoint URL. Um endpoint HTTP seria outra arquitetura e exigiria validar as assinaturas dos pedidos. Fontes oficiais: Gateway e interações.

2 Criar a aplicação e obter as credenciais

Abre o Discord Developer Portal, inicia sessão e cria uma aplicação chamada Último Round. Se o portal apresentar os termos de desenvolvimento, lê-os e aceita-os apenas se concordares. No onboarding, a finalidade deste projeto é Build a Bot.

Em General Information, copia o Application ID. É o identificador público da aplicação, não uma palavra-passe. Em Bot, encontra o token: o portal pode exigir Reset Token para gerar um valor novo. Esse valor autentica o bot e deve ser guardado apenas localmente ou num gestor de segredos. Se redefinires o token, o anterior deixa de funcionar.

No Discord, ativa o Modo de programador e usa Copiar ID no servidor Bots Playground. O Guild ID identifica o servidor onde os comandos serão registados. Os IDs devem ser tratados como strings:

não os convertas para Number em JavaScript, porque podem ultrapassar a precisão de um inteiro seguro.

- Application ID: identifica a aplicação e aparece nas rotas REST de registo.
- Bot Token: segredo usado por client.login e pelas chamadas REST do bot.
- Guild ID: identifica o servidor de testes e limita onde esta versão responde.
- Public Key: serve para verificar assinaturas de interações HTTP; não é necessária neste Gateway.
- Client Secret: pertence a fluxos OAuth2 específicos; não é o Bot Token e não é usado aqui.
- Interaction Token: token temporário de uma interação; é diferente do segredo permanente do bot e é tratado pela biblioteca.

Referência oficial: primeiro bot e credenciais. Não publiques tokens em screenshots, commits, mensagens ou logs. Se houver exposição, redefine o token no portal e atualiza o ambiente local.

3 Preparar o ambiente local

Abre um terminal na pasta ultimo-round. O package-lock.json fixa a árvore de dependências e npm ci instala-a. Copia o ficheiro de exemplo e preenche os valores num editor local.

```
npm ci
Copy-Item .env.example .env
```

O .env tem este formato; os valores reais não estão no documento nem no código distribuído:

```
DISCORD_TOKEN=valor_guardado_localmente
DISCORD_APPLICATION_ID=id_da_aplicacao
DISCORD_GUILD_ID=id_do_Bots_Playground
```

O ficheiro .gitignore exclui .env e permite apenas .env.example. O arranque usa o suporte nativo de Node para carregar o ambiente; não precisa da biblioteca dotenv. Nunca uses uma variável VITE_ ou outra exportação para frontend para guardar este token. Este projeto é um processo de backend.

Antes de configurar o Discord, podes executar as partes locais:

```
npm test
npm run demo
npm run check:source
```

Os testes usam dados fictícios e não fazem pedidos ao Discord. A demonstração gera apenas uma narrativa no terminal. check:source faz duas consultas públicas ao VLR.gg para confirmar que o parser ainda funciona.

4 Instalação contextos permissões e intents

Na página Installation da aplicação, usa Guild Install. Os scopes necessários são bot e applications.commands. Para esta versão, as permissões de servidor previstas são View Channels, Send Messages e Embed Links. Não é necessário Administrator. O comando npm run invite produz uma ligação com essas opções e sem qualquer token secreto; a instalação deve ser autorizada no Bots Playground por quem pode gerir o servidor.

Scopes, permissões e intents resolvem problemas diferentes. Scopes dizem o que a instalação autoriza. Permissões controlam o que o bot pode fazer num servidor ou canal. Intents selecionam os eventos enviados pelo Gateway. Um bloqueio de Embed Links no canal, por exemplo, pode impedir a resposta com embed mesmo que o registo de comandos esteja correto.

O código usa apenas GatewayIntentBits.Guilds. Não lê mensagens normais, presenças nem listas completas de membros. Não ative Message Content Intent, Server Members Intent ou Presence Intent para tentar corrigir um problema de slash commands. A seleção de pessoas em /clutch já chega nos dados resolvidos da interação.

```
const client = new Client({
  intents: [GatewayIntentBits.Guilds],
  allowedMentions: { parse: [], repliedUser: false }
});
```

Referências: permissões e Gateway intents.

5 Registar os slash commands

src/commands.js descreve o contrato público com SlashCommandBuilder. As opções de utilizador são resolvidas pelo Discord, em vez de o bot tentar interpretar nomes escritos em mensagens. /clutch exige jogador1 e jogador2 e aceita um mapa opcional; os comandos competitivos aceitam equipa e foco.

```
npm run register
npm start
```

O script scripts/register-commands.js usa REST v10 e Routes.applicationGuildCommands. Faz um POST por comando para criar ou atualizar o mesmo nome nesse servidor, preservando outros comandos da aplicação. Não faz bulk overwrite nem publica comandos globais. A documentação recomenda comandos de guild para testes, pois as atualizações ficam disponíveis imediatamente nesse âmbito.

```
await rest.post(
  Routes.applicationGuildCommands(appId, guildId),
  { body: command }
);
```

Se um comando desaparecer do código, este script não o apaga do Discord. A remoção explícita futura deve usar o endpoint DELETE do comando. Referência: Application Commands.

6 Executar e responder a interações

src/index.js cria o Client, valida o ambiente e encaminha InteractionCreate para src/handler.js. O handler rejeita outros servidores, aplica cinco segundos de intervalo por utilizador e escolhe a função do comando. Para respostas de erro ou ajuda, usa uma mensagem efémera, visível apenas a quem executou o comando.

O Discord exige a primeira resposta em três segundos. Como uma consulta externa pode demorar mais, /jogos e /resultado chamam deferReply antes da rede e editReply depois. O token temporário de interação tem uma janela de utilização de quinze minutos, mas não substitui o prazo inicial. Não deves chamar reply duas vezes para a mesma interação.

```
await interaction.deferReply();
const snapshot = await source.get('upcoming');
await interaction.editReply({
  embeds: [matchEmbed('upcoming', snapshot)]
});
```

reply cria a primeira resposta. deferReply confirma que vais responder depois. editReply altera essa resposta. followUp seria uma mensagem adicional. O projeto não imprime erros brutos ou payloads de interação, porque podem conter tokens ou dados de quem usou o comando. Referência: Receiving and Responding.

7 A lógica dos clutches

src/clutch.js é independente do Discord. Recebe duas pessoas, escolhe vencedor, mapa e uma situação escrita previamente. O sorteio usa crypto.randomInt; não consulta ranks ou histórico real. A função aceita um seletor injetável para os testes poderem provar os dois resultados sem depender do acaso.

O código rejeita a mesma pessoa nos dois lados e utilizadores bot. As histórias são identificadas como ficção. Os IDs aparecem em referências de utilizador na mensagem, mas allowedMentions desliga as notificações por menção. Os mapas são cenários narrativos; a lista não pretende representar a rotação competitiva atual.

Para personalizar, acrescenta situações à lista scenes e frases à lista endings. Mantém cada história coerente: se a pessoa defende e ganha o duelo após a spike ser plantada, a narrativa deve explicar o defuse quando isso for necessário para ganhar o round. Não introduzas resultados reais na lista de ficção.

8 Ler os jogos e apresentar informação fiável

src/vlr.js consulta as páginas públicas de calendário e resultados do VLR.gg. Cheerio interpreta o HTML e extrai as equipas, bandeiras, pontuações, evento, fase, data e ligação da partida. Não é uma API oficial da Riot nem do VLR; a estrutura da página pode mudar. A API comunitária vlrggapi foi avaliada, mas o seu README indicava indisponibilidade do alojamento público durante a pesquisa.

A fonte tem cache em memória de dez minutos por página e partilha pedidos simultâneos. O pedido tem timeout de oito segundos. Uma falha de rede, HTTP inválido ou estrutura inesperada produz indisponibilidade, em vez de gerar dados. Os resultados apresentados incluem crédito e data da consulta.

O foco Brasil usa a bandeira da equipa ou o nome do evento, não a nacionalidade de cada jogador. Uma equipa explícita substitui o foco. O recorte é a página atual: não encontrar um jogo não prova que não existem outros. Os horários são os exibidos pela fonte, com fuso não confirmado; esta versão não faz conversões sem um timestamp fiável.

Fontes: calendário VLR, resultados VLR, estado da API comunitária. Um produto público maior deve reavaliar as condições de utilização, a cobertura e a estabilidade da fonte escolhida.

9 Testes e organização do projeto

- src/config.js valida os IDs e a presença do token sem o imprimir.
- src/commands.js define o contrato dos comandos.
- src/handler.js controla a execução e o ciclo das respostas.
- src/clutch.js contém as regras e narrativas de ficção.
- src/vlr.js isola a dependência da fonte externa.
- src/format.js constrói embeds e mensagens de ajuda.
- scripts/register-commands.js regista os comandos só quando o executas.
- tests/bot.test.js verifica sorteio, dados, cache, filtros, erros e limites.

Os testes são executados com o runner nativo de Node. As fixtures HTML são fictícias e não devem aparecer como resultados reais. Existe um teste que impede o filtro LOUD de confundir o nome com a sequência de letras em Cloud9. O handler é testado para confirmar que deferReply ocorre antes da consulta à fonte.

O teste real deve começar no Bots Playground com /ajuda, depois /clutch escolhendo duas pessoas distintas. Aguarda cinco segundos entre comandos. Testa /jogos com foco Internacional e /resultado com uma equipa. Confirma embeds, indicação de fonte e resposta quando não há correspondências. Nesta execução, a ligação Gateway e os comandos /ajuda e /clutch foram confirmados no Bots Playground. O calendário e os resultados foram verificados na fonte e nos testes locais; convém experimentá-los também no Discord.

10 Diagnóstico e próximos passos

Se os comandos não aparecem, confirma o Application ID e o Guild ID, o scope applications.commands e o resultado de npm run register. Se o bot está offline, verifica se npm start continua aberto e se o token é válido. Um erro de intent não deve ser corrigido ativando permissões adicionais ao acaso: esta versão usa só Guilds.

Se o Discord mostra que a aplicação não respondeu, verifica se o processo está ativo e se uma alteração fez trabalho lento antes de deferReply. Se só os dados competitivos falham, executa npm run check:source; /clutch deve continuar independente dessa fonte. Se um embed falha num canal, verifica as permissões efetivas nesse canal.

Para manter o bot online, será necessário um processo Node persistente e segredos configurados no alojamento. O Gateway não funciona simplesmente colocando estes ficheiros no Worker HTTP do portefólio. Para já, Ctrl+C termina o bot e nada é guardado numa base de dados. Notícias, perguntas livres e atualizações automáticas são evoluções futuras, não funcionalidades desta versão.

Referências para continuar a aprender

- Documentação oficial do Discord
- Primeiro bot e configuração da aplicação
- Gateway
- Slash commands
- Ciclo das interações
- Permissões de servidor e canal
- Documentação do discord.js

Este documento é um guia do projeto, não uma tradução integral da documentação oficial. Os nomes e limites foram consultados em 19 de outubro de 2026; consulta as referências quando evoluíres a aplicação.